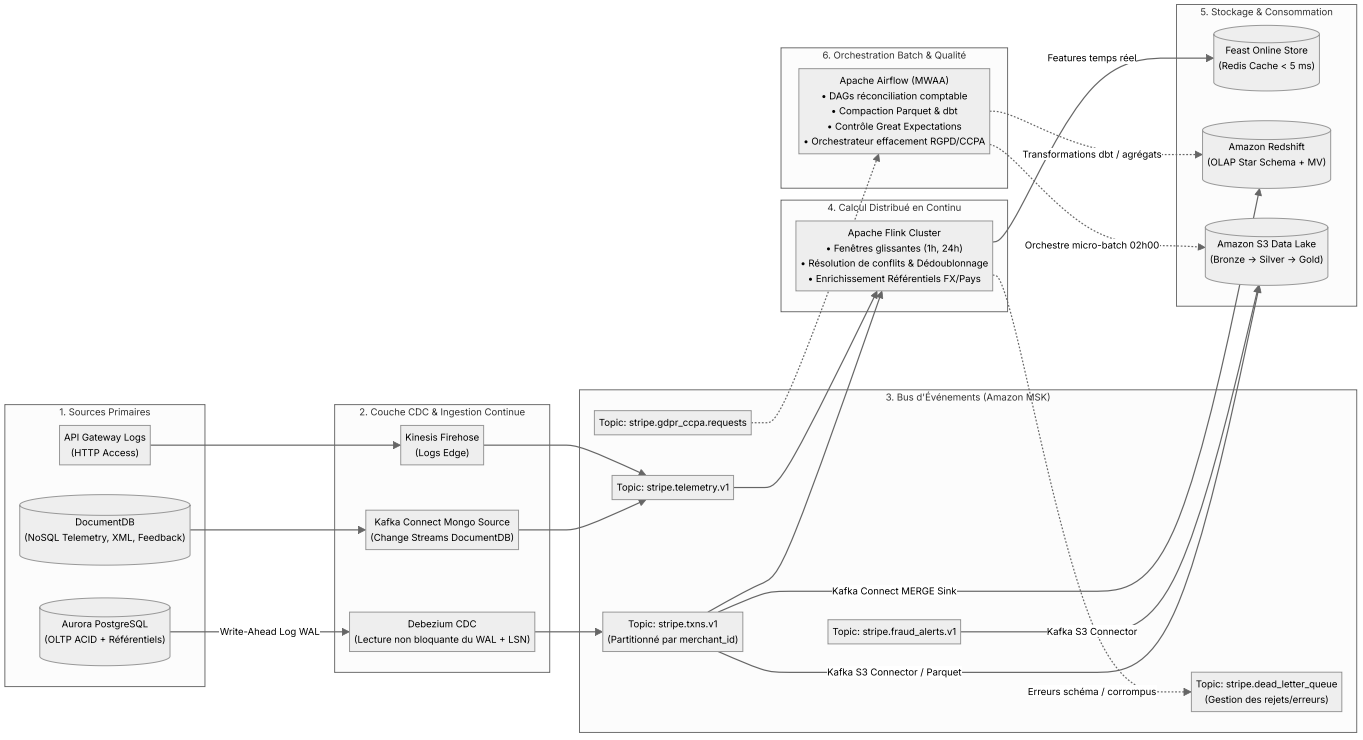
STRIPE AI INFRASTRUCTURE • CERTIFICATION BLOC 2 (RNCP 41993)

Architecture d'Intégration & Pipelines de Données — CDC, Streaming, Batch & Cohérence

Chez Stripe, le plus grand défi d'ingénierie consiste à faire communiquer des systèmes ayant des contraintes temporelles opposées : - **Temps Réel (< 100 ms) :** Le moteur de scoring Radar et Adaptive Checkout doivent consommer les dernières transactions et traces matérielles sans attendre. - **Streaming Continu (< 1 seconde) :** Les commits validés dans l'OLTP doivent être répliqués vers le bus d'événements sans impacter les performances de la base de production. - **Analytique & Décisionnel (1 à 5 minutes) :** Le Data Warehouse (Redshift) doit être alimenté en continu pour les dashboards marchands, les alertes fraude et la surveillance de conformité. - **Entraînement de Modèles & Archivage (Batch quotidien/hebdomadaire) :** Les téraoctets d'historique et de données de référence sont consolidés au format Parquet dans le Data Lake S3.

2. Diagramme Détaillé du Pipeline de Données



3. Matrice Complète des Flux du Pipeline

Source	Format Source	Outil d'Extraction	Fréquence / Fraîcheur	Traitement / Transformation	Cible	Format Cible
Aurora PostgreSQL (Transactions & Abonnements)	Binaire WAL (PostgreSQL)	**Debezium CDC** (Connecteur Kafka)	Continu (< 500 ms)	Filtrage des champs sensibles, conservation du `LSN`, sérialisation Avro avec Schema Registry	**Amazon MSK** (`stripe.txns.v1`)	Apache Avro (Compressé Snappy)
Amazon DocumentDB (Clickstream, XML, Feedback)	BSON / JSON	**Kafka Connect Mongo Source**	Quasi temps réel (< 1s)	Normalisation UTC, masquage PII, extraction scores NLP et empreinte `fingerprint_hash`	**Amazon MSK** (`stripe.telemetry.v1`)	JSON / Avro
Amazon MSK (Flux fusionné + Référentiels)	Avro / JSON	**Apache Flink** (Stream Processing)	Temps réel (< 200 ms)	Jointure temporelle (*Event-Time*), fenêtres glissantes 1h/24h, conversion FX uniforme	**Feast / Redis Cache**	Clé-Valeur binaire Redis (TTL 24h)
Amazon MSK (`stripe.txns.v1`)	Avro	**Kafka Connect Redshift Sink**	Micro-batch (toutes les 2 min)	Dédoublonnage idempotent (`MERGE`) via `transaction_id` & `lsn`, chargement en colonnes	**Amazon Redshift** (`FactTransactions`)	Colonnes compressées AZ64
Amazon MSK (Tous les topics)	Avro	**Kafka S3 Connector**	Streaming (Flush 100 Mo / 15 min)	Partitionnement par date (`/year=YYYY/month=MM/day=DD/`), écriture tabulaire	**S3 Data Lake (Silver)**	Apache Parquet (Compression Snappy)
S3 Silver & Référentiels	Parquet & API FX	**Apache Airflow + dbt**	Batch Quotidien (02h00 UTC)	Contrôle Great Expectations, réconciliation comptable, calcul RFM et vues mensuelles	**Redshift & S3 Gold**	Tables OLAP agrégées & Datasets ML

4. Principes Clés d'Ingénierie des Données

4.1. Pourquoi le Change Data Capture (CDC) avec Debezium ?

Dans une architecture de paiement, il est formellement proscrit d'exécuter des requêtes récurrentes de type `SELECT * FROM transactions WHERE updated_at > :last_sync` sur la base transactionnelle : 1. **Zéro Impact de Performance :** Debezium lit directement le journal des écritures binaires (*Write-Ahead Log* ou WAL) d'Aurora au niveau du stockage sans solliciter le moteur SQL. 2. **Capture Intégrale :** Les requêtes SQL périodiques manquent les états transitoires rapides (`authorized → captured → refunded`). Le CDC capture **100% des événements d'écriture** avec état *before/after*.

4.2. Idempotence & Sémantique *Exactly-Once*

Dans un système distribuant des milliards d'euros, un message Kafka dupliqué lors d'une coupure réseau ne doit jamais créer un double comptage analytique : - **Partitionnement par Clé Métier :** Le topic `stripe.txns.v1` est partitionné par `merchant_id`, garantissant l'ordre chronologique strict au sein d'un même compte marchand. - **Dead Letter Queue (DLQ) :** Tout message au schéma Avro invalide ou rejeté est isolé dans `stripe.dead_letter_queue` avec alerte PagerDuty, évitant tout blocage de consommation (*backpressure*).

4.3. Orchestration Batch & Qualité avec Apache Airflow (MWAA)

Chaque nuit à 02h00 UTC, un DAG Airflow orchestre la chaîne ETL/ELT : 1. **Validation de Schéma (Great Expectations) :** Bloque le pipeline en cas de valeurs nulles critiques, taux de change manquants ou déséquilibre comptable (`gross ≠ stripe_fee + net`). 2. **Transformations ELT (dbt) :** Alimente les dimensions (SCD Type 2) et agrège `FactDailyMerchantRevenue` et `AggMonthlyProductPerformance`. 3. **Compaction Parquet :** Fusionne les micro-fichiers issus du streaming en blocs optimisés (128 à 512 Mo) pour accélérer Redshift Spectrum, Athena et SageMaker.

5. Modèles de Cohérence (ACID vs Cohérence Éventuelle) & Résolution de Conflits

Une architecture polyglotte distribuée doit arbitrer explicitement entre cohérence immédiate et disponibilité/débit :

5.1. Matrice des Niveaux de Cohérence par Système

Système	Modèle de Cohérence	Fenêtre de Convergence	Justification d'Ingénierie
Aurora PostgreSQL (OLTP)	**Cohérence Forte (ACID / Linearizability)**	**0 ms (Synchrone)**	Obligatoire pour l'argent : un solde, un statut de paiement ou une clé d'idempotence doivent être immédiatement vrais partout.
Feast / Redis (Online Store)	**Cohérence Éventuelle Bornée**	**< 500 ms**	Acceptable pour des compteurs de vélocité fraude glissants ; tolère une milliseconde d'écart en échange d'une lecture RAM en < 5 ms.
Amazon DocumentDB (NoSQL)	**BASE / Cohérence Éventuelle (Read Preference Secondary)**	**< 1 seconde**	Priorise la haute disponibilité d'ingestion pour la télémétrie clickstream et les logs massifs sans verrous globaux.
Amazon Redshift (OLAP)	**Cohérence Éventuelle Analytique**	**1 à 5 minutes (Micro-batch)**	La BI décisionnelle n'exige pas la milliseconde ; le regroupement en micro-batchs évite la fragmentation mémoire du stockage colonnaire.

5.2. Stratégies de Résolution de Conflits Distribués

En environnement distribué (rejeu réseau Kafka, redémarrage Flink, réplication inter-services), trois garde-fous éliminent les conflits d'état :

1. **Ordonnancement Déterministe par LSN (*Log Sequence Number*) :**
   Chaque événement CDC extrait par Debezium embarque le `txid` et le `lsn` exacts du journal WAL d'Aurora. En cas d'arrivée désordonnée sur un consommateur ou de reprise sur incident, la comparaison numérique des `lsn` prime toujours sur l'horloge locale des machines (*clock skew immunity*).
2. **Machine à États Monotone sur les Statuts Financiers :**
   Dans les pipelines de fusion (`MERGE` Redshift & Flink), une transition d'état ne peut jamais régresser (ex : `authorized [1] → captured [2] → succeeded [3] → disputed [4] → refunded [5]`). Si un rejeu tardif apporte l'état `authorized [1]` alors que `succeeded [3]` est déjà enregistré, l'écriture obsolète est ignorée (*Conflict Guard*).
3. **Dédoublonnage Idempotent par Clé Composite (`MERGE / UPSERT`) :**
   Côté Redshift (`staging → fact_transactions`) et S3 Silver, l'intégration exécute un `MERGE` atomique ne mettant à jour une ligne existante que si `source.lsn > target.lsn` (stratégie *Last-Write-Wins* déterministe sur séquence WAL).